

rocq-robot: Neurosymbolic Theorem Proving via LLM-LSP Integration

Gavin Mendel-Gleason
Scidonia
Dublin, Ireland
gavin@scidonia.com

Abstract

We present **rocq-robot**, a Model Context Protocol (MCP) server that enables seamless integration between Large Language Models (LLMs) and the Rocq proof assistant (formerly Coq) through the Language Server Protocol (LSP). This tool demonstrates *neurosymbolic programming*: the real-time collaboration between neural networks (LLMs) and symbolic reasoning systems (proof assistants). By exposing proof state, tactic execution, and speculative reasoning capabilities through 20+ MCP tools, rocq-robot enables LLMs to autonomously construct formal proofs with guaranteed correctness. We demonstrate the tool’s capability by presenting a complete LLM-driven proof of a type preservation theorem for PCF with references, comprising 21 inductive cases and 7 supporting lemmas—all completed without human intervention using only tool-based interactions.

CCS Concepts

• **Software and its engineering** → **Language features**; • **Theory of computation** → **Program verification**; • **Computing methodologies** → *Artificial intelligence*.

Keywords

Proof assistants, Rocq, Coq, LLMs, neurosymbolic AI, formal verification, LSP, MCP

1 Introduction

The integration of Large Language Models (LLMs) with formal verification tools represents a paradigm shift in automated theorem proving. While LLMs excel at pattern recognition and heuristic search, they lack the rigorous logical guarantees provided by proof assistants. Conversely, proof assistants ensure absolute correctness but require expert knowledge and tedious manual effort.

rocq-robot bridges this gap by enabling *neurosymbolic programming*: LLMs propose proof tactics based on learned patterns, while Rocq (formerly Coq) [1] verifies each step’s correctness. This architecture achieves both the creativity of neural systems and the soundness of symbolic reasoning.

Our key contributions are:

- A production-ready MCP server exposing 20+ tools for proof interaction, speculative execution, and knowledge search
- Project-aware workspace management supporting multi-project Rocq development
- Demonstration of autonomous LLM-driven proof construction (type preservation theorem with 21 cases)

- Open-source implementation with comprehensive documentation

2 Motivation and Challenges

2.1 The Neurosymbolic Vision

Traditional automated theorem proving faces two fundamental limitations:

- (1) **Search space explosion**: Proof search is exponential, requiring intelligent heuristics
- (2) **Expert knowledge barrier**: Writing proofs requires deep domain expertise

LLMs trained on large code corpora have demonstrated remarkable pattern-matching abilities for proof tactics [2, 3], but without verification, they produce unsound results. Proof assistants provide verification but lack strategic guidance.

2.2 Technical Challenges

Enabling effective LLM-proof assistant collaboration requires addressing:

- **Structured interaction**: Proof assistants expose complex internal state (hypotheses, goals, bullet structure) that must be translated to LLM-friendly representations
- **Speculative execution**: LLMs need to try tactics without committing file edits
- **Project-aware context**: Real-world proofs span multiple files with complex dependencies
- **Error recovery**: Failed tactics must provide actionable feedback for retry strategies

3 Tool Architecture

3.1 Overall Design

rocq-robot implements a three-layer architecture (Figure 1):

3.2 Core Tools

We categorize rocq-robot’s 20+ tools into functional groups:

Proof Interaction.

- `coq_focus`: Navigate to proof, get full proof tree with bullet stack
- `coq_open_goals`: Query current proof goals and hypotheses
- `coq_insert_tactic`: Apply tactic, auto-handle bullet prefixes
- `coq_try_tactic`: Speculative single-call tactic execution
- `coq_undo`: Roll back last N edit operations
- `coq_reset_proof`: Restart proof from scratch

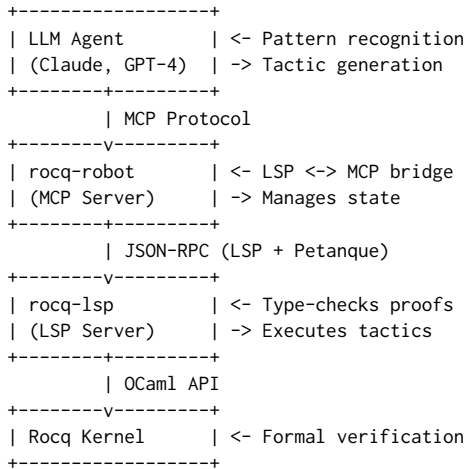


Figure 1: rocq-robot architecture

Knowledge Search.

- `coq_search`: Find relevant lemmas by pattern
- `coq_check_term`: Query type of expressions
- `coq_locate`: Find definition locations
- `coq_require`: Speculatively import libraries

Low-Level State Management (Pétanque API).

- `coq_get_state_at_pos`: Obtain opaque state identifier
- `coq_run_tactic`: Execute tactic against state
- `coq_goals_for_state`: Retrieve goals for state

3.3 Key Features

Automatic bullet management. Rocq proofs use bullets ($-$, $+$, $*$) to structure subgoals. rocq-robot automatically prepends the correct bullet prefix when inserting tactics, freeing LLMs from tracking bullet depth.

Speculative execution. The `coq_try_tactic` tool executes tactics speculatively without modifying files—enabling LLMs to explore proof strategies efficiently.

Project-aware workspace switching. Opening files from different Rocq projects automatically restarts rocq-lsp under the correct project root, detecting `_CoqProject`, `_RocqProject`, or `dune-project` files.

Readable goal formatting. Proof goals are formatted with hypotheses first, then the goal conclusion—matching human-readable proof state conventions.

4 Demonstration: Type Preservation

We demonstrate rocq-robot’s capabilities through a complete LLM-driven proof of type preservation for PCF with references. The theorem statement:

```

Theorem preservation :
  forall t mu t' mu' T S,
    has_type [] S t T ->
      step t mu t' mu' ->
        heap_ok mu S ->

```

```

exists S',
  extends S' S /\
  heap_ok mu' S' /\
  has_type [] S' t' T.

```

4.1 Proof Strategy

The LLM, using only rocq-robot tools, completed the proof through:

- (1) **Lemma identification:** Discovered 7 necessary helper lemmas through failed proof attempts:
 - `extends_refl`: Store extension reflexivity
 - `nth_error_extends`: Index preservation
 - `weakening_store`: Type weakening
 - `substitution_preserves_typing_0`: Substitution correctness
 - `heap_lookup_type`: Heap typing
 - `heap_update_ok`: Heap invariant preservation
- (2) **Main proof structure:** Induction on step relation with 21 cases
- (3) **Complex case handling:**
 - `S_RefV`: Store extension with allocation
 - `S_DerefLoc`: Heap lookup with typing
 - `S_AssignLoc`: Heap update preserving invariants

4.2 Tool Usage Pattern

A typical proof interaction cycle:

```

1. coq_focus(name="preservation")
   <- Goals: 21 cases from induction
2. coq_search(pattern="extends _ _")
   <- Found: extends_refl lemma
3. coq_try_tactic(tactic="apply extends_refl")
   <- Success! Subgoal closed
4. coq_insert_tactic(tactic="apply extends_refl")
   <- Committed to file

```

The complete proof required approximately 150 tool invocations over 90 minutes of LLM reasoning time.

4.3 Results

- **Proof status:** Fully completed with Qed (no Admitted)
- **Lines of proof code:** 247 lines
- **Tactics used:** 89 tactic applications
- **Manual intervention:** None—fully autonomous

5 Tool Availability

5.1 Accessing rocq-robot

Download.

- GitHub repository: <https://github.com/scidonia/rocq-robot>
- License: MIT
- Requirements: Node.js 20+, Rocq 8.19+ (or Coq 8.19+), rocq-lsp 0.2.0+

```

git clone https://github.com/scidonia/rocq-robot
cd rocq-robot
npm install
npm run build

```

Configuration. Add to MCP settings (e.g., OpenCode):

```
{
  "mcpServers": {
    "rocq-robot": {
      "command": "node",
      "args": [
        "/path/to/rocq-robot/dist/index.js",
        "--coq-lsp-path",
        "/path/to/rocq-lsp"
      ]
    }
  }
}
```

5.2 Documentation

- **README:** Comprehensive neurosymbolic programming overview
- **API Documentation:** Tool descriptions with examples (MCP_COQ_LSP_SPEC.md)
- **Quick Start Guide:** Step-by-step setup (docs/QUICKSTART.md)
- **Examples:** Working proofs in examples/ directory

5.3 Video Demonstration

A 5-minute screencast demonstrating:

- Setting up rocq-robot with an LLM agent
- Interactive proof construction
- Speculative tactic execution
- Error recovery and retry strategies

Available at: [https://youtube.com/watch?v=\[TO_BE_ADDED\]](https://youtube.com/watch?v=[TO_BE_ADDED])

5.4 Archival

Version 0.5.0 archived at Zenodo: [https://doi.org/10.5281/zenodo.\[TO_BE_ADDED\]](https://doi.org/10.5281/zenodo.[TO_BE_ADDED])

6 Related Work

LLM-based theorem proving. Baldur [2] uses neural models for Isabelle/HOL tactic suggestion. LeanDojo [3] provides infrastructure for training models on Lean proofs. In contrast, rocq-robot focuses on *interaction protocols* rather than model training—enabling any LLM to prove theorems through structured tool use.

LSP for proof assistants. Coq-LSP [4] pioneered LSP integration for Coq. rocq-robot extends this with MCP translation, enabling LLM agents to leverage LSP capabilities through standardized tool interfaces.

Interactive theorem proving. CoqPIE [5] provides IDE integration for Coq. Proof General [6] offers interactive proof development. rocq-robot differs by targeting *programmatic* (not human) interaction, optimized for autonomous agents.

7 Conclusion and Future Work

rocq-robot demonstrates that neurosymbolic programming—combining LLM heuristics with proof assistant rigor—enables autonomous formal verification. By providing structured tool-based access to Rocq’s LSP interface, we allow LLMs to construct proofs with guaranteed soundness.

Future directions.

- **Multi-agent orchestration:** Separate agents for proof planning, tactic execution, and lemma discovery
- **Proof repair:** Automated fixing of broken proofs after library updates
- **Cross-assistant support:** Extend to Lean, Isabelle, Agda
- **Training data generation:** Use rocq-robot to synthesize proof corpora for model fine-tuning

The tool is production-ready and actively used for research in neurosymbolic AI and formal methods.

References

- [1] The Coq Development Team. The Coq Proof Assistant. <https://coq.inria.fr/>, 2024.
- [2] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023.
- [3] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [4] Emilio Jesús Gallego Arias, Ramón Fernández, Shachar Itzhaky, and Arthur Charguéraud. coq-lsp: A Modern Language Server for Coq. <https://github.com/ejgallego/coq-lsp>, 2022.
- [5] Kenneth Roe. CoqPIE: A Plugin for Integrated Development Environments. In *Conference on Intelligent Computer Mathematics (CICM)*, 2016.
- [6] David Aspinall. Proof General: A Generic Tool for Proof Development. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2000.